

Smog Forecast Agent — 4-Day Crash Roadmap (Sept 26 → Sept 30 submission)

Reality check: Original 20-day plan is dead. This is now a 2-person sprint over 4 real build days + 1 submission day. Multi-city expansion is OFF — Delhi-NCR only. No more 30-min sessions — this needs real hours each day from both of you.

Hard dependency: the model must exist by end of Day 2 (Sept 27). Everything downstream (agent, dashboard, backtest) depends on it. Protect that date above all else.

Today — Sept 26 (You: finish data pipeline. Partner: continue agent on mock data.)

You:

- Finish loading ground_observations + satellite_features into BigQuery (Day 6-7 tasks)
- **Skip the full date-realignment re-download if you're short on time today.** Instead: check whatever overlap currently exists between your CPCB, Mendeley, and Earth Engine data and use that window, even if it's just a few overlapping months. A smaller correct dataset beats a bigger broken one, and you don't have time to redo downloads.
- Build the combined `training_data` table (the SQL join from Phase 3 Day 8) using whatever real overlap you have.
- **Kick off AutoML Forecasting training tonight**, even if the table isn't perfect — training takes hours, so start it before you stop for the day. If BigQuery data isn't ready by evening, prioritize getting even a rough version of `training_data` in place so the training job can run overnight.

Partner (Phase 4, in parallel):

- Keep building the Gemini agent against **mock forecast output** (hand-write 3-4 example rows: district, predicted AQI, severity, feature importances) so the prompt and logic are ready regardless of whether the real model finishes on time.
 - Get the threshold-to-category mapping logic done (India's AQI bands).
 - Draft and test the alert-generation prompt in AI Studio using the mock data.
-

Day 2 — Sept 27 (Model must land today. Agent gets wired to it.)

You:

- Check the AutoML job. If it succeeded, note feature importances immediately — partner needs these for the "likely cause" reasoning in the alert.

- If AutoML failed, is still running past a reasonable point, or the metrics look bad: **switch to the fallback trend model as your primary model, right now.** Do not keep troubleshooting AutoML into Day 3 — a working simple model beats a broken sophisticated one.
- Once you have a working model (either one), generate real forecast output for a handful of dates/stations and hand it to your partner.

Partner:

- Swap the mock data for real model output. Confirm the agent's alert drafting still works end-to-end with real numbers (not just the hand-crafted examples).
- Add the Hindi translation step if not already done.
- **By end of today, you should have:** forecast → threshold check → Gemini-drafted bilingual alert, running as one connected pipeline, even if rough.

Day 3 — Sept 28 (Dashboard + interoperability config + multi-city snapshot)

Both of you, split the work:

- **Person A:** Build the Maps Platform dashboard — Delhi-NCR districts color-coded by forecasted severity, clickable to show the drafted alert.
- **Person B:** Build `region_config.json` (thresholds, schema field names) AND pull "snapshot mode" data for Mumbai, Kolkata, Bengaluru:
 - Download one 2017-2023 hourly station CSV per city from OpenCity (same format as Delhi's, same reshape script works unchanged)
 - Run each through `fallback_forecast_model.py` (AQI-only is fine — if wind_speed isn't available, the script's wind_factor just defaults to neutral, no code change needed for that)
 - Feed each city's forecast through the same Gemini agent to generate an alert
 - Add each city as an additional marker on the dashboard, using `region_config.json` to show the schema is genuinely shared, not hardcoded per city
- **Time-box this hard: 3-4 hours max for all three cities combined.** If a city's download is being difficult (bad format, station data doesn't load cleanly), cut it and move on — two solid extra cities beats three rushed, broken ones.
- End of day: Delhi dashboard should pull from real (or backtest) pipeline output, not hardcoded values. Other cities can be a simpler "current snapshot" view rather than a full forecast history.

Day 4 — Sept 29 (Backtest, pitch, rehearsal — no new features after today)

- **Morning:** Pick one real historical Delhi smog spike date from your data. Run the pipeline "as if" forecasting 48h before it. Screen-record this — it's your strongest demo moment and your insurance if something breaks live.
 - **Midday:** Write the pitch narrative (2 min): problem → live map lighting up → Gemini alert in Hindi → backtest proof → interoperability config as the scalability answer.
 - **Afternoon:** Full dry run, timed, both of you. Fix only things that are actually broken — resist the urge to add anything new today.
 - **Evening:** Freeze the build. Nothing changes after tonight except submission logistics.
-

Sept 30 — Submission day

- No new building. This day is for: final upload, checking submission requirements are met exactly as specified, and having a buffer in case the platform/portal has issues.
-

Non-negotiable cuts, given the timeline

- **Mumbai/Kolkata/Bengaluru are AQI-only snapshot cities, not full deep pipelines.** Delhi-NCR is the only city with weather + satellite + AutoML training. Don't attempt to replicate the full pipeline for the other three — there isn't time, and a broken attempt is worse than an honest, clearly-labeled lighter demo.
- **If snapshot-mode data sourcing runs long on Day 3, cut a city rather than cut into Day 4.** Two extra cities that work is a stronger demo than three that are half-broken.
- **No live telephony/SMS integration.** Simulated alert routing to a mock dashboard is fine.
- **AutoML is not sacred.** If it's not working well by Day 2, the fallback trend model becomes your real model. A judge will not penalize a simpler model that works over a sophisticated one that's broken or absent.
- **Speaker notes, polish, and animations** on the dashboard are the first thing to cut if Day 3 runs long. A plain, working map beats a pretty, broken one.